

Spatial transcriptomics

Tutorial Links

Explore the following tutorials to get started with spatial transcriptomics analysis:

- Spatial Transcriptomics Toolkit [semla](#). In addition to this tutorial, there are two other parts of this notebook that were derived, covering the extraction and analysis of spatial transcripts in the same package.
- [Seurat Spatial Transcriptomics Vignette](#) How to perform spatial transcriptomics analysis with the Seurat package?

Install the packages

```
In [ ]: ## Function to execute terminal commands in Google Colab with the R kernel
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...) # Executes the terminal command
  cat(paste0(result, collapse = "\n")) # Prints the output in a readable format
}
# Downloads the code from the specified URL and saves it as "add_cranapt_jammy.sh"
download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_cranapt_jammy.sh",
              "add_cranapt_jammy.sh")
Sys.chmod("add_cranapt_jammy.sh", "0755")
shell_call("./add_cranapt_jammy.sh")
bspm::enable()
options(bspm.version.check=FALSE)
shell_call("rm add_cranapt_jammy.sh")
# Sets a higher timeout limit to avoid interruptions during package downloads.
options(timeout=1000)
```

```
In [ ]: # Install the "semla" package from GitHub, forcing an update.
remotes::install_github("ludvigla/semla", upgrade=T, force = TRUE)
```

```
In [ ]: # Install packages
cranPkgs2Install = c("BiocManager", "ggpubr", "Seurat", "hdf5r", "openxlsx",
                    "enrichR", "clustree", "DT", "ggcorrplot", "scatterpie", "pheatmap")
install.packages(cranPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)
```

```
In [ ]: biocPkgs2Install = c("SingleCellExperiment", "ReactomePA", "org.Hs.eg.db", "glmGamPari")
BiocManager::install(biocPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)
```

```
In [ ]: # Load packages
# Suppress messages when loading packages to keep the output clean.
suppressPackageStartupMessages({
  library(Seurat) # Framework for single-cell RNA-seq analysis.
  library(data.table) # Efficient manipulation of large datasets.
  library(ggplot2) # Visualization based on the grammar of graphics.
  library(plotly) # Interactive visualizations.
  library(RColorBrewer) # Predefined color palettes.
```

```
library(dplyr) # Data manipulation.
library(semLa) # Tools for spatial transcriptomics.
library(clustree) # Visualization of clustering resolutions.
library(ReactomePA) # Pathway analysis.
library(org.Hs.eg.db) # Human gene annotation database.
library(ggpubr) # Publication-ready visualizations.
library(enrichR) # Gene enrichment analysis.
library(stringr) # String manipulation.
library(openxlsx) # Excel file manipulation.
library(patchwork) # Combine multiple plots.
library(SingleCellExperiment) # Structure for single-cell data.
})
```

Introduction

Spatial transcriptomics is a technique that combines information about gene expression with the physical location of cells within a tissue. Unlike traditional transcriptomic methods, here we know not only which genes are expressed, but also where they are expressed, allowing for a more precise study of cellular organization and tissue microenvironments.

In this context, histological images play a fundamental role. One of the most widely used is H&E (Hematoxylin and Eosin) staining, which stains tissues to highlight their structure:

- Hematoxylin stains cell nuclei blue/purple.
- Eosin stains the cytoplasm and extracellular components pink.

The combination of spatial transcriptomics with H&E imaging allows for the correlation of molecular profiles with tissue architecture, offering a comprehensive view of how cells are organized and function in their natural environment.

Download the data

The data downloaded below corresponds to a spatial transcriptomics dataset available on the 10x website and generated with Space Ranger (<https://www.10xgenomics.com/support/software/space-ranger/latest>). The sample is cancerous breast tissue.

You can find details at the following links:

- [Human Breast Cancer \(Block A, Section 1\)](#)
- [Human Breast Cancer \(Block A, Section 2\)](#)

```
In [ ]: shell_call("mkdir -p ST_Exercises")

# Download the files containing spatial transcriptomics exercises
download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca
download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca
download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca
```

```

download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca
download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca
download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca
download.file('https://cf.10xgenomics.com/samples/spatial-exp/1.1.0/V1_Breast_Ca

# List the files in the current directory with their sizes
shell_call("ls -lh ST_Exercises")

```

```

In [ ]: shell_call("tar -xvzf ST_Exercises/Breast_Cancer_Block_A_Section_1_spatial.tar.g
shell_call("mv ST_Exercises/spatial ST_Exercises/Spatial_Section_1")
shell_call("tar -xvzf ST_Exercises/Breast_Cancer_Block_A_Section_2_spatial.tar.g
shell_call("mv ST_Exercises/spatial ST_Exercises/Spatial_Section_2")

```

Load the dataset

```

In [ ]: samples <- list.files(
  path = "/content/ST_Exercises",
  pattern = "filtered_feature_bc_matrix.h5",
  full.names = TRUE,
  recursive = TRUE
)

imgs <- list.files(
  path = "/content/ST_Exercises",
  pattern = "tissue_lowres_image.png",
  full.names = TRUE,
  recursive = TRUE
)

spotfiles <- list.files(
  path = "/content/ST_Exercises",
  pattern = "positions_list.csv",
  full.names = TRUE,
  recursive = TRUE
)

json <- list.files(
  path = "/content/ST_Exercises",
  pattern = "scalefactors_json.json",
  full.names = TRUE,
  recursive = TRUE
)

infoTable <- tibble(samples, imgs, spotfiles, json, # Add required columns
  sample_id = c("Breast_Cancer_Block_A_Section_1", "Breast_Can
dim(infoTable)

```

```

In [ ]: # Load matrix
SpatialData <- ReadVisiumData(infoTable)
SpatialData

```

```

In [ ]: # Load a Seurat object.
# Load(file = "ST_Exercises/Exercise1_dataset.RData")
# Get dataset information

```

```
# This command retrieves the current default assay of the Seurat object SpatialData
DefaultAssay(SpatialData)

# Shows the number of genes (features) and cells.
dim(SpatialData)
head(SpatialData@meta.data)

# This command generates a frequency table of the "sample_id" column
# in the metadata of the Seurat object SpatialData.
table(SpatialData@meta.data$sample_id)
```

Quality Control (QC) plots

Plot the metrics

```
In [ ]: # Violin plot with mean values added.
VlnPlot(Her2p, features = "nCount_RNA", group.by = "ids",
        pt.size = 0.1, cols = color) +
  stat_summary(fun.y=mean, geom="point", shape=95,
              size=15, color = "black") + NoLegend()

# Scatter plot comparing RNA counts to the number of features (genes).
FeatureScatter(object = Her2p, feature1 = "nCount_RNA",
               feature2 = "nFeature_RNA", group.by = "ids", cols = color)

# Overlays RNA count distribution on spatial coordinates.
FeatureOverlay(Her2p, features = c("nCount_RNA"),
               sampleids = 1:3,
               pt.size = 2.5,
               add.alpha = TRUE,
               ncol = 3, min.cutoff = 0, max.cutoff = 2000)
```

```
In [ ]: # Scatter plot comparing RNA quantity with the number of features (genes)
FeatureScatter(object = SpatialData, feature1 = "nCount_Spatial",
               feature2 = "nFeature_Spatial", group.by = "sample_id", cols = col
```

Exercise: Save the image to a file

```
In [ ]: # Load H&E images
SpatialData <- LoadImages(SpatialData)
ImagePlot(SpatialData)
```

```
In [ ]: # Mapping tissue cells
MapFeatures(SpatialData, features = "nFeature_Spatial",
            image_use = "raw", override_plot_dims = TRUE) & ThemeLegendRight()
```

```
In [ ]: MapFeaturesSummary(
  object = SpatialData,      # your semla/Seurat object
  features = "nCount_Spatial", # metric to be displayed
  pt_size = 2.5,             # point size
  ncol = 2,                  # number of columns in the layout
  subplot_type = "histogram" # type of plot
)
```

Seurat's alternative for graphically representing spatial transcriptomics data.

```
In [ ]: # Create a copy to work with
SpatialData2 <- SpatialData

# Load files into a slot
SpatialData2@images[["slice1"]] <- Read10X_Image(dirname(SpatialData2@tools$Staf
SpatialData2@images[["slice2"]] <- Read10X_Image(dirname(SpatialData2@tools$Staf

# Visualize the data
SpatialFeaturePlot(SpatialData2, features = "nCount_Spatial") +
  theme(legend.position = "right")

# Remove the copy
rm(SpatialData2)
```

Normalization is a crucial preprocessing step in scRNA-seq data analysis. It involves adjusting raw gene expression data to compensate for technical variations and differences in sequencing depth between cells. These comparisons should help understand the strengths and limitations of each normalization method.

Normalization Method

- SCTransform: A more advanced method that uses negative binomial regression to remove technical variability and stabilize variance, providing robust normalization for single-cell data.
- Logarithmic Normalization: Scales gene expression values and applies a logarithmic transformation, which helps stabilize variance and makes the data more suitable for further analysis.

Removal of Technical Variability

- SCTransform: Greater robustness in removing technical variability, making it ideal for datasets with significant variation.
- NormalizeData: Effective, but less sophisticated than SCTransform in handling technical variability.

Complexity and Execution Time

- SCTransform: More complex and laborious due to regression calculations and variance stabilization transformation.
- NormalizeData: Simpler and faster, ideal for quick analyses and large datasets.

Covariate Handling

- SCTransform: Allows the inclusion of covariates (e.g., number of genes detected per cell) for regression, which improves the quality of normalized data.

- `NormalizeData`: Typically does not incorporate covariates directly into the normalization process.

Variance Stabilization

- `SCTransform`: Stabilizes variance, making the data more suitable for subsequent analyses such as clustering and marker identification.
- `NormalizeData`: Uses logarithmic transformation to partially stabilize variance, but not as effectively as `SCTransform`.

Flexibility

- `SCTransform`: Focuses primarily on handling single-cell RNA sequencing data with robust normalization methods.
- `NormalizeData`: Flexible with different normalization methods (e.g., `LogNormalize`), allowing adjustments based on specific analysis needs.

Scalability

- `SCTransform`: Can handle large datasets, but may be slower due to their complexity.
- `NormalizeData`: Handles large datasets efficiently thanks to its simplicity and speed.

```
In [ ]: # Create a UMAP plot grouped by "ids", without cluster labels
color = getPalette(9) # Generates a vector of 9 colors using the getPalette func
DimPlot(Her2p, reduction = "umap", label = FALSE, group.by = "ids",
        pt.size = 2, cols=color, label.size=13)
```

```
In [ ]: # Plot only the slide 3 of the patient G
# Overlay the expression of one or more features onto spatial coordinates.
FeatureOverlay(Her2p, # The Seurat object containing your data.
               features = c("SCT_snn_res.0.5"), # Specifies the features to over
               sampleids = 2, # Indicates that only the slide with ID 2 should b
               pt.size = 3.5,
               cols = color,
               add.alpha = TRUE, # Add transparency to points
               ncol = 1)
```

```
In [ ]: # Plot only the 3 slide of the patient G
FeatureOverlay(Her2p, features = c("SCT_snn_res.0.4"),
               sampleids = 1:3, # Use slides 1 to 3
               pt.size = 3.5,
               cols = color,
               add.alpha = TRUE,
               ncol = 3) # Arrange plots in 3 columns
```

```
In [ ]: # Normalize data using LogNormalize with a scaling factor
SpatialData <- NormalizeData(object = SpatialData, assay = "Spatial")
```

```
In [ ]: SpatialData <- FindVariableFeatures(SpatialData, nfeatures = 10000)
```

The following comparison takes a considerable amount of time; it is **recommended not to run it**.

```
In [ ]: # Normalize data using SCTransform
SpatialData <- SCTransform(SpatialData, assay = "Spatial", verbose = TRUE, returnData = TRUE)

# Compare normalization methods (LogNormalize vs SCTransform)
SpatialData <- GroupCorrelation(SpatialData, group.assay = "Spatial", assay = "Spatial")
SpatialData <- GroupCorrelation(SpatialData, group.assay = "Spatial", assay = "SCT")

# Generate comparison plots
p1 <- GroupCorrelationPlot(SpatialData, assay = "Spatial", cor = "nCount_Spatial")
p2 <- GroupCorrelationPlot(SpatialData, assay = "SCT", cor = "nCount_Spatial_cor")

# Show both plots side by side
p1 + p2
```

Clustering

```
In [ ]: # Install package by GitHub
devtools::install_github("zdebruine/RcppML")
devtools::install_github("zdebruine/singlet")
```

```
In [ ]: # Load package
library(singlet)

# Set seed for reproducibility
set.seed(42)
SpatialData2 <- SpatialData

# OPTIONAL: create a subset of data to improve calculation speed
SpatialData2 <- SpatialData2[VariableFeatures(SpatialData2), ]
SpatialData2 <- RunNMF(SpatialData2)
```

```
In [ ]: # Transfer the reductions
SpatialData@reductions <- SpatialData2@reductions

# Remove the copy
rm(SpatialData2)
gc()

# Transfer the columns
k <- ncol(SpatialData@reductions$nmf@feature.loadings)
k
```

`RankPlot(SpatialData)` creates a figure that shows the ranking of NMF factors according to their contribution, helping to select the most relevant ones for interpreting spatial patterns of expression.

```
In [ ]: # RankPlot creates a figure with a ranking data graph.
RankPlot(SpatialData)
```

We will see how the expression patterns are spatially distributed in the tissue, which facilitates the interpretation of cellular heterogeneity.

```
In [ ]: # Adjust the dimensions of the plots to be generated
options(repr.plot.width=12, repr.plot.height=6)

# First block of NMF features (from 1 to 6)
MapFeatures(SpatialData,
            features = paste0("NMF_", 1:6),           # Selects NMF_1 to NMF_
            override_plot_dims = TRUE,                 # Forces the dimensions
            colors = viridis::magma(n = 11, direction = -1)) & # Applies the in
            theme(plot.title = element_blank())        # Removes the plot title

# Second block of NMF features (from 7 to 12)
MapFeatures(SpatialData,
            features = paste0("NMF_", 7:12),          # Selects NMF_7 to NMF_
            override_plot_dims = TRUE,
            colors = viridis::magma(n = 11, direction = -1)) &
            theme(plot.title = element_blank())

# Third block of NMF features (from 13 to 18)
MapFeatures(SpatialData,
            features = paste0("NMF_", 13:18),         # Selects NMF_13 to NMF_
            override_plot_dims = TRUE,
            colors = viridis::magma(n = 11, direction = -1)) &
            theme(plot.title = element_blank())

# Fourth block of NMF features (from 19 to k)
MapFeatures(SpatialData,
            features = paste0("NMF_", 19:k),          # Selects NMF_19 up to
            override_plot_dims = TRUE,
            colors = viridis::magma(n = 11, direction = -1)) &
            theme(plot.title = element_blank())
```

In this step, we created a feature loading plot derived from the NMF reduction. This type of visualization allows us to identify which genes contribute most strongly to the early latent components, displaying the most relevant ones in a scatter plot. This facilitates the biological interpretation of spatial patterns and the understanding of how each gene participates in the variability captured by the NMF.

```
In [ ]: # Generate a feature loadings plot
# for the spatial data using NMF reduction

PlotFeatureLoadings(SpatialData,
                    dims = 1:2,           # Selects the first two dimensions (com
                    reduction = "nmf",    # Specifies that the reduction used is
                    nfeatures = 30,       # Number of most relevant features (gen
                    mode = "dotplot",     # Sets the visualization mode as a dot
                    fill = "darkmagenta", # Fill color for the points in the plot
                    pt_size = 3)          # Point size in the visualization
```

In this step, a multi-characteristic map of NMFs is generated over the spatial data. This function allows the simultaneous visualization of all latent components (NMF_1 to NMF_k) projected onto the raw tissue image. We adjusted the dimensions of the graphs and the size of the points to obtain a clear and uniform representation, facilitating the interpretation of how each expression pattern is spatially distributed within the tissue context.

```
In [ ]: # Adjust the dimensions of the plots to be generated
options(repr.plot.width=16, repr.plot.height=9)

# Generate a multiple-feature map of NMF components in the spatial data
MapMultipleFeatures(SpatialData,
  features = paste0("NMF_", 1:k), # Selects all NMF features from 1 to k
  image_use = "raw",               # Uses the raw image (unprocessed) as
  override_plot_dims = TRUE,       # Forces the dimensions defined above
  pt_size = 2)                    # Sets the size of the points in the
```

In this visualization, we seek to understand which genes are primarily responsible for each latent component generated by NMF reduction. By visualizing the trait burdens as a heat map, we can clearly compare how the contribution of genes varies across different dimensions.

```
In [ ]: # Generate a heatmap of feature loadings
# using NMF reduction in the spatial data

PlotFeatureLoadings(SpatialData,
  dims = 1:k,           # Selects all dimensions from 1 to k
  reduction = "nmf",    # Specifies that the reduction used is NMF
  nfeatures = 5,        # Shows the 5 most relevant features per
  mode = "heatmap",     # Sets the visualization mode as heatmap
  gradient_colors = viridis::magma(n = 11, # Applies the 'magma'
                                     direction = -1)) # Inverted
```

```
In [ ]: # Plot UMAP
SpatialData <- RunUMAP(SpatialData, reduction = "nmf", dims = 1:k, verbose = FALSE)
```

```
In [ ]: # Perform graph-based clustering with Seurat
# Find the nearest neighbors for each cell using Non-negative Matrix Factorization
SpatialData <- FindNeighbors(object = SpatialData,
  dims = 1:k, # Uses the first k dimensions of the specified
  reduction = "nmf", # Indicates that NMF was used as the dimensionality
  verbose = FALSE) # Suppresses detailed output

# Define a sequence of resolutions from 0 to 1.2, with increments of 0.2
SpatialData <- FindClusters(SpatialData, resolution = seq(0, 1.2, by = 0.2),
  verbose = FALSE)
```

```
In [ ]: head(SpatialData@meta.data)
```

```
In [ ]: # Visualize clustering results at different resolutions using Clustree
clustree(SpatialData, prefix = "Spatial_snn_res.")
```

```
In [ ]: # Create a UMAP plot grouped by "sample_id", without cluster labels
DimPlot(SpatialData, reduction = "umap", label = FALSE, group.by = "sample_id",
  pt.size = 2, label.size=13)
```

```
In [ ]: SpatialData2 <- SpatialData
SpatialData2@images[["slice1"]] <- Read10X_Image(dirname(SpatialData2@tools$Staf
SpatialData2@images[["slice2"]] <- Read10X_Image(dirname(SpatialData2@tools$Staf

# Plot only slice 3 of patient G
# Overlay the expression of features on spatial coordinates
SpatialDimPlot(SpatialData2, group.by = "Spatial_snn_res.0.4") + theme(legend.po
```

```
MapMultipleFeatures(SpatialData2,
                    features = paste0("NMF_", 1:k), # Selects all NMF features f
                    image_use = "raw",             # Uses the raw tissue image
                    override_plot_dims = TRUE,      # Forces the dimensions defi
                    pt_size = 2)                   # Sets the point size in the
```

Exercise: Save the image to a file.

Visualize diretamente a distribuição de aglomerados no espaço tecidual, sem interferência da imagem histológica de fundo.

```
In [ ]: # Plot clusters without the histological (HE) image background
SpatialDimPlot(SpatialData2, group.by = "Spatial_snn_res.0.4", image.alpha = 0)
  theme(legend.position = "right") # Visualize clusters defined at resolution

# Add Labels to clusters on the spatial map
MapLabels(SpatialData, column_name = "Spatial_snn_res.0.4", ncol = 2) &
  theme(legend.position = "right") # Place the legend on the right for easier
```

Differentially Expressed Genes

In this part of the analysis, differentially expressed marker genes (DEGs) are identified among the different spatial clusters. Filtering by statistical significance ensures that only reliable genes are retained, and the final DEG count per cluster allows us to assess which cell populations exhibit more defined or enriched molecular profiles.

```
In [ ]: devtools::install_github('immunogenomics/presto')
```

```
In [ ]: DefaultAssay(SpatialData) # Check the active assay of the Seurat SpatialData obj
Idents(SpatialData) <- "Spatial_snn_res.0.4" # Set the identity of the object ac

# Compare all clusters against each other
SpatialData.markers <- FindAllMarkers(object = SpatialData, # Identify different
                                     only.pos = TRUE, # Only genes overexpressed in c
                                     min.pct = 0.10, # Gene must be expressed in at L
                                     logfc.threshold = 0.10) # Minimum log2 fold chan

# Filter markers with adjusted p-value < 0.05
SpatialData.markers = SpatialData.markers[which(SpatialData.markers$p_val_adj <

# Count the number of DEGs per cluster
table(SpatialData.markers[, "cluster"]) # How many DEGs exist per cluster?
```

```
In [ ]: # Select the top 10 genes with highest logFC per cluster
SpatialData.markers %>% group_by(cluster) %>% top_n(n = 10, wt = avg_log2FC) ->

# Generate a dot plot for the selected genes
DotPlot(SpatialData, features = unique(top10$gene),
        group.by = "Spatial_snn_res.0.4", cols = c('#b8d8d8', '#e71d36'),
        dot.scale = 6, col.min = 0) +
  theme(axis.text.x = element_text(face = "bold", color = c("black"),
    size = 8, angle = 90))
```

Exercise: Save the image to a file.

This section shows how expression varies between different groups, helping to identify which cell populations are enriched in these genes, thus facilitating direct comparison of the intensity and frequency of expression between groups.

```
In [ ]: # Plot key genes on a density plot (ridge plot)
RidgePlot(Her2p, assay = "SCT",
          features = c("IFI27", "IFI6"), # Genes de interés
          ncol = 2, group.by = "SCT_snn_res.0.4",
          cols = color)

In [ ]: # UMAP visualization and heatmap
# Define the color gradient for the heatmap
heatmap.colors <- c("lightgray", "mistyrose", "red", "darkred", "black")
fts <- c("SPAG6", "PGM5-AS1") # List of genes to be visualized

# Generate UMAP plots for each gene
p.fts <- lapply(fts, function(ftr) {
  FeaturePlot(SpatialData, features = ftr, reduction = "umap", order = TRUE,
             cols = heatmap.colors, pt.size = 2) # Define point size
})

# Plot gene expression mapped to Visium spatial coordinates
p3 <- MapFeatures(SpatialData, features = fts, ncol = 2, cols = heatmap.colors,

# Combine all plots into a single figure
cowplot::plot_grid(cowplot::plot_grid(plotlist = p.fts, ncol = 1), p3, ncol = 2,

In [ ]: # Violin plot for expression of SPAG6 and PGM5-AS1 grouped by clusters
# in the Seurat object 'SpatialData', grouped by clustering resolution "Spatial_
VlnPlot(SpatialData, features = c("SPAG6", "PGM5-AS1"),
        group.by = "Spatial_snn_res.0.4", # Defines the grouping variable (clust
        pt.size = 0, # Sets point size to 0 to avoid plotting individual points
        ncol = 2) + # Arranges plots in two columns

# Overlay a summary statistic (mean) as a horizontal bar
stat_summary(fun = mean, geom = "point", shape = 95,
            size = 15, color = "black") +

# Remove the Legend from the plot
NoLegend()
```

3D visualization

```
In [ ]: library(plotly)
library(dplyr)
library(htmlwidgets)
```

It retrieves the spatial coordinates of each point or cell in the tissue image and organizes them by sample. Simultaneously, it obtains information about the image associated with the object, allowing expression data to be linked to its physical position in the tissue. This is fundamental in spatial transcriptomics, which connects molecular profiles with tissue architecture and facilitates biological interpretation within its spatial context.

```

In [ ]: # Get the spatial coordinates of each spot/cell in the image
xy_coords <- GetCoordinates(SpatialData) |>
  # Note: use pxl_*_in_fullres for the raw image; if sections are aligned,
  # you should use pxl_*_in_fullres_transformed
dplyr::select(pxl_col_in_fullres, pxl_row_in_fullres, sampleID) |>
group_by(sampleID) |>
group_split() # Split coordinates by sample

# Extract information about the image associated with the spatial object
image_info <- GetImageInfo(SpatialData)

In [ ]: # Adjust spatial coordinates of each sample
adjusted_coords <- do.call(bind_rows, lapply(seq_along(xy_coords), function(i) {
  xy <- xy_coords[[i]] # Extract coordinates of sample i
  full_width <- image_info[i, ]$full_width # Get the total width of the image
  full_height <- image_info[i, ]$full_height # Get the total height of the image
  xy <- xy |>
    mutate(x = pxl_col_in_fullres/full_width, # Normalize x coordinate relative
           y = pxl_row_in_fullres/full_height) |> # Normalize y coordinate relative
  # Define a z value using sampleID to separate sections in 3D space.
  # This adjustment allows controlling the distance between aligned sections.
  mutate(z = sampleID*0.2) |>
  dplyr::select(x, y, z) # Keep only the adjusted coordinates
}))

In [ ]: # 3D scatter plot with plotly
p <- plot_ly(adjusted_coords, x = ~x, y = ~y, z = ~z, type = "scatter3d", mode =
saveWidget(as_widget(p), "SpatialData3DPlot.html")

```

Gene Ontology (GO) Terms

```

In [ ]: # Select differentially expressed genes (DEGs) from cluster 6
geneList = SpatialData.markers[which(SpatialData.markers$cluster == 6), "gene"]
length(geneList) # Show the number of selected genes

# Convert gene symbols to Entrez IDs
columns(org.Hs.eg.db) # Show available columns for conversion in the annotation

symbol <- mapIds(org.Hs.eg.db,
  keys = geneList, # List of gene symbols to convert
  column = "ENTREZID", # Convert to Entrez IDs
  keytype = "SYMBOL", # Input type: gene symbol
  multiVals = "first") # If multiple matches, take the first one

symbol = as.vector(symbol) # Convert to vector format

# Remove genes without Entrez ID
geneList = symbol
geneList = geneList[which(geneList != "NA")]
length(geneList) # Show the number of genes after filtering

# Pathway enrichment analysis with ReactomePA
x <- enrichPathway(gene = geneList,
  organism = "human",
  pvalueCutoff = 0.05,
  readable = TRUE,
  pAdjustMethod = "bonferroni")

```

```

head(as.data.frame(x)) # Show the first rows of the results

# Order results by adjusted p-value (from smallest to largest)
x@result = x@result[order(x@result$p.adjust, decreasing = FALSE),]

# Select the 30 most significant pathways
x@result = x@result[1:30,]

# Order pathways by gene count (from smallest to largest)
x@result = x@result[order(x@result$Count, decreasing = FALSE),]

# Save the ordered results in a new data frame
newbar.dt = x@result

# Create a bar plot with enriched pathways
cluster6_enrichpathway <- ggbarplot(newbar.dt, x = "Description", y = "Count",
  fill = "Count",          # Color bars according to the number of genes
  color = "white",         # White border color for bars
  sort.val = "asc",        # Sort values in ascending order
  sort.by.groups = FALSE, # Do not sort by group
  x.text.angle = 90,       # Rotate x-axis labels for better visibility
  xlab = "Pathways",
  ylab = "Number of genes",
  legend.title = "Count",
  rotate = TRUE,
  ggtheme = theme_minimal()
) + scale_fill_continuous(low = "#bde0fe", high = "red") +
  theme(text = element_text(size = 20))

```

Pathways analysis with EnrichR

```

In [ ]: # List available databases in EnrichR
listEnrichrSites()
dbs <- listEnrichrDbs()

# Order databases by name
dbs %>% dplyr::arrange(libraryName)

# Select specific databases for enrichment analysis
dbs <- c("Cancer_Cell_Line_Encyclopedia",
  "Elsevier_Pathway_Collection",
  "KEGG_2021_Human",
  "CellMarker_Augmented_2021",
  "Reactome_2016",
  "GO_Biological_Process_2018",
  "GO_Cellular_Component_2018",
  "GO_Molecular_Function_2018",
  "InterPro_Domains_2019")

# EnrichR requires gene symbols (not Ensembl IDs)
geneList = SpatialData.markers[which(SpatialData.markers$cluster == 6), "gene"]
length(geneList) # Show the number of selected genes

# Perform enrichment analysis with EnrichR
enriched.Paths <- enrichr(geneList, dbs)

# Show results from selected databases

```

```

enriched.Paths[[1]] # Cancer_Cell_Line_Encyclopedia
enriched.Paths[[2]] # Elsevier_Pathway_Collection
enriched.Paths[[3]] # KEGG_2021_Human
enriched.Paths[[4]] # CellMarker_Augmented_2021
enriched.Paths[[5]] # Reactome_2016
enriched.Paths[[6]] # GO_Biological_Process_2018
enriched.Paths[[7]] # GO_Cellular_Component_2018
enriched.Paths[[8]] # GO_Molecular_Function_2018
enriched.Paths[[9]] # InterPro_Domains_2019

# Select results from CellMarker_Augmented_2021
Rpath = enriched.Paths[[4]]

# Count the number of genes per pathway
Gene_Count = c()
for(x in 1:dim(Rpath)[1]){
  Gene_Count = c(Gene_Count, str_split(Rpath$Overlap[x], pattern = "/" )[[1]][1])
}

Rpath$Gene_Count = as.numeric(Gene_Count)

# Order by adjusted p-value (from smallest to largest)
Rpath = Rpath[order(Rpath$Adjusted.P.value, decreasing = FALSE),]

# Filter significant pathways (adjusted p-value < 0.05)
Rpath = Rpath[Rpath$Adjusted.P.value < 0.05,]

# Select the top 20 pathways
Rpath = Rpath[1:20,]

# Order pathways by gene count (from largest to smallest)
Rpath = Rpath[order(Rpath$Gene_Count, decreasing = TRUE),]

# Create a bar plot of enriched pathways
cluster6_enrichR <- ggbarplot(Rpath, x = "Term", y = "Gene_Count",
  fill = "Gene_Count", # Color bars according to gene count
  color = "white", # White border color
  sort.val = "asc", # Sort values in ascending order
  sort.by.groups = FALSE,
  x.text.angle = 90, # Rotate labels for better visibility
  xlab = "Pathways",
  ylab = "Number of genes",
  legend.title = "Count",
  rotate = TRUE,
  ggtheme = theme_minimal()
) + scale_fill_continuous(low = "#bde0fe", high = "red") +
  theme(text = element_text(size = 18))

# Show the plot
cluster6_enrichR

```

Deconvolution of Cell Types

Deconvolution is a computational technique used to infer and quantify the proportions of different cell types in a mixed cell population. Since scRNA-seq data often comes from complex tissues containing multiple cell types, deconvolution helps identify and isolate the specific gene expression profiles of each cell type within the mixture.

Why is deconvolution important?

- Cell type identification: Deconvolution allows researchers to determine the presence and abundance of various cell types in a heterogeneous tissue sample.
- Understanding cellular composition: It helps elucidate the cellular composition of tissues, revealing how different cell types contribute to biological processes and diseases.
- Improved data interpretation: By separating gene expression signals from different cell types, deconvolution improves the accuracy and interpretability of scRNA-seq data, leading to more precise biological insights.

```
In [ ]: download.file('https://github.com/integrativebioinformatics/scNotebooks/blob/mai
```

```
In [ ]: # Load the scRNA-seq dataset
SC.data = readRDS("ST_Exercises/scRNASeq_pac2_processed.rds")
SC.data <- UpdateSeuratObject(SC.data)

# Set cell identities according to cell type
Idents(SC.data) = "cellType"

# UMAP visualization of cell types
DimPlot(object = SC.data, reduction = 'umap', label = TRUE, label.size = 6,
        group.by = "cellType", pt.size = 1.5)

# Identify DEGs across all clusters
SC.markers <- FindAllMarkers(object = SC.data, only.pos = TRUE, min.pct = 0.10,
# min.pct = 0.10: at least 10% of cells must express the gene
# logfc.threshold = 0.10: minimum log2 fold change threshold to consider a gene

SC.markers = SC.markers[which(SC.markers$p_val_adj < 0.05 & SC.markers$avg_log2F
# SC.markers$p_val_adj < 0.05: selects markers with adjusted p-value < 0.05 (sta
# SC.markers$avg_log2FC > 0.5: selects markers with average log2 fold change > 0

# Count the number of DEGs per cluster
table(SC.markers$cluster)

DefaultAssay(SC.data) # returns the name of the assay currently set as default i
```

Deconvolution Pipeline Flow

```
In [ ]: #ti <- Sys.time()
DefaultAssay(SpatialData) <- "Spatial"

# Prediction of cell type proportions
SpatialData <- RunNNLS(object = SpatialData,
                      singlecell_object = SC.data,
                      groups = "cellType")
```

```
In [ ]: # Check the available cell types
rownames(SpatialData)
```

```
In [ ]: # Load H&E images
SpatialData <- SpatialData |>
  LoadImages()
```

```
In [ ]: # Plot multiple features
MapMultipleFeatures(SpatialData,
  image_use = "raw",
  pt_size = 2, max_cutoff = 0.95,
  override_plot_dims = TRUE,
  features = rownames(SpatialData)) +
  plot_layout(guides = "collect")

MapMultipleFeatures(SpatialData,
  pt_size = 2, max_cutoff = 0.95,
  override_plot_dims = TRUE,
  features = rownames(SpatialData)) +
  plot_layout(guides = "collect")
```

Colocalization of Cell Types

Colocalization of cell types refers to the analysis of how different cell populations are distributed and appear together in specific tissue regions. In spatial transcriptomics, this step allows the identification of patterns of proximity or interaction between cell types, revealing potential functional relationships, cell communication, or microenvironments relevant to biological processes and diseases.

```
In [ ]: library(pheatmap)

# Extract the expression matrix from SpatialData and compute correlation between
cor_matrix <- FetchData(SpatialData, rownames(SpatialData)) |>
  mutate_all(~ if_else(.x < 0.1, 0, .x)) |> # Filter very low values, setting to 0
  cor() # Compute correlation matrix

diag(cor_matrix) <- NA # Remove diagonal (self-correlation)
max_val <- max(cor_matrix, na.rm = T) # Get maximum correlation value

# Define color palette for heatmap (reversed red-yellow-blue, with white in the
cols <- RColorBrewer::brewer.pal(7, "RdYlBu") |> rev(); cols[4] <- "white"

# Adjust figure dimensions
options(repr.plot.width = 6, repr.plot.height = 6)

# Generate heatmap of correlations between cell types within spots
pheatmap::pheatmap(cor_matrix,
  breaks = seq(-max_val, max_val, length.out = 100), # Correlation values
  color = colorRampPalette(cols)(100), # Color gradient
  cellwidth = 14, cellheight = 14, # Cell size
  treeheight_col = 10, treeheight_row = 10, # Dendrogram heights
  main = "Correlation between cell types\nwithin spots") # Plot title
```

```
In [ ]: # Applies non-negative matrix factorization (NMF) to spatial data
nmf_data <- FetchData(SpatialData, rownames(SpatialData)) |> # Extract a matrix
  RcppML::nmf(k = 10, verbose = T) # Execute NMF computation
```

```

In [ ]: # Convert the H matrix from NMF result into a data.frame
nmf_data_h <- nmf_data@h |> as.data.frame()

# Assign row names to factors (Factor_1 to Factor_10)
rownames(nmf_data_h) <- paste0("Factor_", 1:10)

# Assign column names corresponding to cells/spots from the spatial object
colnames(nmf_data_h) <- rownames(SpatialData)

# Normalize values of each column by dividing by the maximum (range between 0 and 1)
nmf_data_h <- nmf_data_h |>
  mutate_at(colnames(nmf_data_h),
    ~ (scale(., center = FALSE, scale = max(., na.rm = TRUE))/1)))

# Create a Factor column with row names, keeping the order of factors
nmf_data_h$Factor <- rownames(nmf_data_h) |>
  factor(levels = paste0("Factor_", 1:10))

# Reshape data into Long format:
# each row represents the weight of a factor in a specific cell
nmf_data_h_df <- nmf_data_h |>
  tidyr::pivot_longer(cols = all_of(rownames(SpatialData)),
    names_to = "Cell",
    values_to = "Weight")

```

```

In [ ]: # Bubble chart
ggplot(nmf_data_h_df, aes(x=Factor, y=Cell, size=Weight, color=Weight)) +
  geom_point() +
  labs(title="Cell type contribution", x="Factor", y = "Cell type",
    color = "", size = "Scaled weight") +
  scale_color_viridis_c(direction = -1, option = "magma") +
  theme_bw() +
  theme(axis.text.x = element_text(angle=45, hjust=1),
    panel.grid = element_blank())

```